

Dokumentacja

Biometria

Projekt 1 - Przetwarzanie obrazów

Bogumiła Okrojek

27 marca 2025

Spis treści

1	Opis aplikacji	2
1.1	Wykorzystane technologie i struktura	2
1.2	Interfejs użytkownika	2
1.3	Uzasadnienie wyboru technologii	3
2	Opis metod	3
2.1	Operacje na pikselach	3
2.1.1	Konwersja do odcieni szarości	4
2.1.2	Korekta jasności	5
2.1.3	Korekta kontrastu	6
2.1.4	Negatyw	8
2.1.5	Binaryzacja	8
2.2	Filtry graficzne	10
2.2.1	Filtr uśredniający	10
2.2.2	Filtr Gaussa	12
2.2.3	Filtr wyostrzający	13
2.2.4	Wykrywanie krawędzi - krzyż Robertsa	16
2.2.5	Wykrywanie krawędzi - operator Sobela	17
2.2.6	Możliwość tworzenia dowolnych wag filtrów	18
2.3	Wykresy	19
2.3.1	Histogram jasności	19
2.3.2	Histogram kolorów	20
2.3.3	Projekcja pozioma	20
2.3.4	Projekcja pionowa	21
2.3.5	Projekcja pozioma i pionowa	22
2.4	Inne	22
2.4.1	Wczytanie obrazu z pliku	22
2.4.2	Zapisu pliku przetworzonego	23
2.5	Podsumowanie	24

1 Opis aplikacji

Aplikacja powstała jako narzędzie do przetwarzania obrazów. Jej głównym celem jest demonstracja operacji graficznych, takich jak konwersja do odcieni szarości, regulacja jasności i kontrastu, binaryzacja, generowanie negatywu obrazu oraz stosowanie prostych filtrów, np. filtrów uśredniających.

1.1 Wykorzystane technologie i struktura

Aplikacja została zbudowana w języku Python przy użyciu biblioteki `Tkinter` jako głównego interfejsu graficznego użytkownika (GUI). Wykorzystuje również inne kluczowe moduły i biblioteki, takie jak:

- `matplotlib` – do wizualizacji obrazów i wyświetlania wyników operacji na płótnie,
- `Pillow` (`PIL`) – do obsługi i modyfikacji obrazów w formatach takich jak JPEG, PNG, BMP,

Program podzielono na główne komponenty:

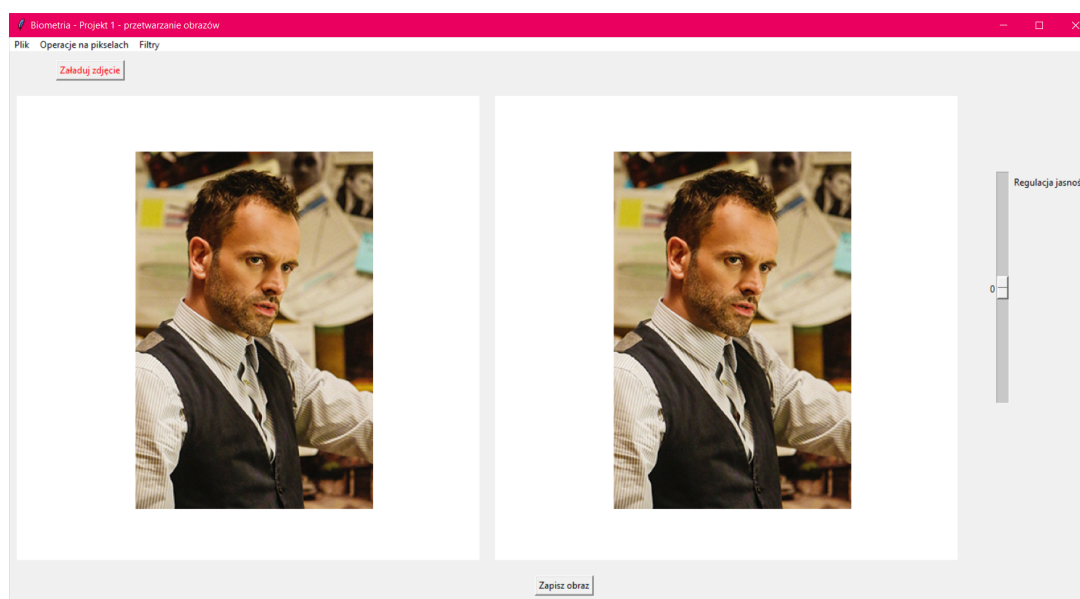
- **Interfejs główny** – aplikacja opiera się na głównym oknie utworzonym za pomocą klasy `Tk()` z biblioteki `Tkinter`. Główne elementy interfejsu to menu z operacjami graficznymi oraz miejsce na wyświetlanie obrazów za pomocą płótna `matplotlib`.
- **Moduł funkcji przetwarzania obrazu** – zbiór funkcji takich jak `change_grey`, `brighthness_change` czy `average_filter`, które modyfikują obraz poprzez operacje matematyczne na macierzach pikseli.
- **Obsługa zdarzeń GUI** – działania takie jak wczytywanie i zapisywanie obrazów są realizowane przy użyciu modułu `filedialog`, który integruje obsługę plików z interfejsem użytkownika.

1.2 Interfejs użytkownika

Interfejs jest zaprojektowany w sposób umożliwiający łatwe korzystanie z funkcjonalności aplikacji. Główne elementy interfejsu widoczne na Rysunek 1 to:

- Menu składające się z trzech części:
 - plik (`Plik`), gdzie możliwe jest załadowanie obrazu z wybranego pliku (`Zaladuj plik`)

- operacje na pikselach (**Operacje na pikselach**), do wyboru są różne przekształcenia (np. Konwersja do odcieni szarości, Korekta jasności)
- filtry (**Filtry**), do wyboru są różne filtry graficzne (np. Filtr uśredniający, Filtr wyostrzający)
- Dwa panele, jeden wyświetlający oryginalny obraz wczytany z pliku, a drugi przekształcony obraz poprzez wybraną operację
- Elementy sterujące, takie jak suwak do regulacji wartości (jasność, kontrast) oraz przyciski radiowe do wyboru rozmiaru maski dla filtrowania obrazu.



Rysunek 1: GUI aplikacji

1.3 Uzasadnienie wyboru technologii

Wybrano język Python ze względu na jego szeroką dostępność bibliotek do przetwarzania obrazu oraz łatwość integracji z narzędziami GUI, takimi jak **Tkinter**. Zastosowane biblioteki, tj. **matplotlib**, **Pillow** i **numpy**, zapewniają wydajność oraz elastyczność w realizacji zadań związanych z obrazami.

2 Opis metod

2.1 Operacje na pikselach

Operacje na pikselach są fundamentalnym elementem przetwarzania obrazów, pozwalającym na ich modyfikację, analizę i poprawę jakości na poziomie pojedynczych punktów składających się

na obraz.

Obraz cyfrowy reprezentowany jest jako macierz pikseli, gdzie każdy piksel w formacie kolorowym opisany jest za pomocą składowych (R, G, B) , odpowiadających wartościom kanałów czerwonego, zielonego i niebieskiego.

Głównym założeniem operacji na pikselach jest ich punktowy charakter – wartości każdego piksela mogą być przetwarzane niezależnie, bez wpływu sąsiednich pikseli. Takie podejście pozwala na implementację podstawowych zadań, takich jak zmiana jasności, kontrastu, konwersja do odcieni szarości czy binaryzacja.

2.1.1 Konwersja do odcieni szarości

Konwersja obrazu do odcieni szarości jest jedną z podstawowych operacji w przetwarzaniu obrazów, której celem jest przekształcenie kolorowego obrazu na obraz w odcieniach szarości, gdzie każdy piksel reprezentuje poziom jasności.

Idea i metoda

W tej operacji następuje redukcja wymiarów – wartość każdego piksela w odcieniach szarości obliczana jest jako średnia arytmetyczna wartości z kanałów czerwonego, zielonego i niebieskiego:

$$S = \frac{R + G + B}{3}$$

gdzie S to jasność piksela w obrazie w odcieniach szarości. Wartość S zostaje przypisana do wszystkich trzech kanałów, tworząc wizualizację szarości w macierzy trójkanałowej, co widać na 3, gdy porównamy go z oryginalnym obrazem 2.



Rysunek 2: Oryginalny obraz



Rysunek 3: Obraz po przekształceniu do odcieni szarości

Wady i ograniczenia

Metoda korzystająca ze średniej arytmetycznej ma swoje ograniczenia:

- Nie odzwierciedla dokładnie percepcji ludzkiego oka, które jest bardziej wrażliwe na kolor zielony niż czerwony czy niebieski. Dla bardziej realistycznych rezultatów można zastosować wzór ważony:

$$S = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

- Efekt końcowy może nie oddawać wszystkich subtelnych zmian w bardzo ciemnych lub jasnych obrazach, można to poprawić poprzez dostosowanie kontrastu za pomocą normalizacji histogramu

2.1.2 Korekta jasności

Korekta jasności to podstawowa operacja, której celem jest globalna zmiana poziomu jasności obrazu, tj. jego rozjaśnienie lub przyciemnienie.

Idea i metoda

Ta operacja polega na dodaniu stałej wartości $c \in [-255, 255]$ do wszystkich trzech składowych RGB każdego piksela obrazu:

$$R = R + c, \quad G = G + c, \quad B = B + c$$

Nowe wartości w pikselu mogą się nie mieścić w zakresie wartości pikseli obrazu RGB: $[0, 255]$, więc wszystkie wyliczone wartości są ucinane do tego zakresu:

$$R = \max(0, \min(R + c, 255)), \quad G = \max(0, \min(G + c, 255)), \quad B = \max(0, \min(B + c, 255))$$

Ta metoda pozwala rozjaśnić oraz pociemnić obrazy, dzięki wyborze wartości dla stałej c tak, że

- Gdy $c < 0$, to obraz się staje ciemniejszy,
- Gdy $c = 0$, to obraz się nie zmienia,
- Gdy $c > 0$, to obraz się rozjaśnia.

Na Rysunek 4, Rysunek 5 i Rysunek 6 wyraźnie widać efekty tej operacji. Dla $c = -80$ (Rysunek 4), że obraz jest znacznie ciemniejszy w porównaniu do obrazu oryginalnego (Rysunek 5), natomiast dla $c = 80$ (Rysunek 6) obraz jest widocznie rozjaśniony.

W aplikacji wartość c może być dynamicznie ustawiana przez użytkownika za pomocą suwaka znajdującego się w interfejsie graficznym po prawej stronie.



Rysunek 4: Obraz po zmianie jasności dla $c = -80$



Rysunek 5: Oryginalny obraz



Rysunek 6: Obraz po zmianie jasności dla $c = 80$

Wady i ograniczenia

Chociaż korekta jasności jest prosta i skuteczna, ma pewne wady:

- Może prowadzić do utraty szczegółów w jasnych lub ciemnych obszarach obrazu.
- Operacja ta jest globalna – zmiana jasności jest stosowana do całego obrazu, co może być niewystarczające, jeśli różne obszary obrazu wymagają różnych poziomów jasności. Aby to poprawić można zastosować lokalne metod korekty jasności, które uwzględniają potrzeby różnych obszarów obrazu (np. techniki adaptacyjne).

2.1.3 Korekta kontrastu

Korekta kontrastu to operacja, której celem jest zmiana różnicy jasności pomiędzy ciemnymi i jasnymi obszarami obrazu. Poprawa kontrastu pozwala na wyraźniejsze uwypuklenie szczegółów obrazu.

Idea i metoda

W tej operacji wartości wszystkich trzech składowych RGB każdego piksela są modyfikowane poprzez podniesienie ich do stałej potęgi a :

$$R = R^a, \quad G = G^a, \quad B = B^a$$

gdzie a to parametr określający poziom kontrastu. Może on przyjmować wartości Po modyfikacji mogą wystąpić sytuacje, w których wynikowe wartości R, G, B przekraczają dopuszczalny zakres.

Aby tego uniknąć, każda wartość jest ograniczana do przedziału zakresu $[0, 255]$:

$$R = \max(0, \min(R^a, 255)), \quad G = \max(0, \min(G^a, 255)), \quad B = \max(0, \min(B^a, 255))$$

Wartość parametru a wpływa na poziom kontrastu:

- Dla $a > 1$: kontrast zostaje zwiększony, co prowadzi do większego uwypuklenia różnic w jasności.
- Dla $a = 1$: obraz pozostaje niezmienny.
- Dla $0 < a < 1$: kontrast zostaje zmniejszony, co powoduje wyrównanie różnic w jasności.

Na Rysunek 7, Rysunek 8 i Rysunek 9 wyraźnie widać efekty tej operacji. Dla $a = \frac{1}{3}$ (Rysunek 7), że obraz ma znacznie mniejszy kontrast niż obrazu oryginalnego (Rysunek 8), natomiast dla $a = 3$ (Rysunek 9) obraz ma widocznie większy kontrast.

Parametr a może być dynamicznie regulowany przez użytkownika za pomocą suwaka dostępnego w interfejsie aplikacji po prawej stronie.



Rysunek 7: Obraz po zmianie kontrastu dla $a = \frac{1}{3}$



Rysunek 8: Oryginalny obraz



Rysunek 9: Obraz po zmianie kontrastu dla $a = 3$

Wady i ograniczenia

Chociaż korekta kontrastu jest prostą i skuteczną metodą, posiada pewne ograniczenia:

- Przy dużych wartościach $a > 1$ może dojść do utraty informacji w najjaśniejszych i najciemniejszych obszarach obrazu.
- Wartości a z zakresu $0 < a < 1$ mogą spowodować zbyt jednolitą jasność obrazu, co zmniejsza jego dynamikę i czytelność.

2.1.4 Negatyw

Tworzenie negatywu to podstawowa operacja, której celem jest stworzenie odwrotności obrazu. Polega ona na odwróceniu intensywności kolorów każdego piksela, co w praktyce oznacza, że najciemniejsze piksele stają się najjaśniejsze, a najjaśniejsze – najciemniejsze.

Idea i metoda

Operacja negatywu jest realizowana przez obliczenie różnicy pomiędzy maksymalną wartością intensywności dla każdego kanału (255) a aktualną wartością danego piksela. Dla każdego kanału RGB (czerwony, zielony, niebieski) obliczenia są wykonywane zgodnie z poniższymi wzorami:

$$R = 255 - R, \quad G = 255 - G, \quad B = 255 - B$$

W ten sposób każdy piksel staje się dopełnieniem swojego oryginału.

Na Rysunki 10 and 11 przedstawiono porównanie między oryginalnym obrazem a jego negatywem. Wyraźnie widać, że odwrócone kolory uwydatniają różnice między jasnymi i ciemnymi obszarami obrazu.



Rysunek 10: Oryginalny obraz



Rysunek 11: Negatyw

2.1.5 Binarizacja

Binarizacja jest podstawową operacją przetwarzania obrazów, której celem jest uproszczenie obrazu poprzez przedstawienie go w dwóch wartościach: czerni i bieli.

Idea i metoda

Binarizacja polega na przypisaniu każdemu pikselowi wartości 0 lub 255 na podstawie tego,

czy jego jasność jest większa czy mniejsza od stałego progu odcięcia T :

$$P = \begin{cases} 255, & \text{jeśli } I = \frac{R+G+B}{3} \geq T \\ 0, & \text{jeśli } I = \frac{R+G+B}{3} < T \end{cases}$$

Na Rysunek 12, Rysunek 13, Rysunek 14 i Rysunek 15 wyraźnie widać efekty binaryzacji dla różnych wartości progu T . Na podstawie porównania z oryginalnym obrazem (Rysunek 12) można zauważyć, że odpowiedni dobór wartości progu ma kluczowe znaczenie dla zachowania szczegółów i poprawności segmentacji.



Rysunek 12: Oryginalny obraz



Rysunek 13: Obraz po binaryzacji dla $T = 50$



Rysunek 14: Obraz po binaryzacji dla $T = 100$



Rysunek 15: Obraz po binaryzacji dla $T = 150$

Dla $T = 50$ (Rysunek 13) obraz zawiera zbyt wiele białych pikseli, co skutkuje utratą szczegółów w jaśniejszych partiach obrazu.

Dla $T = 150$ (Rysunek 15) sytuacja jest odwrotna – na obrazie dominuje czerń, a szczegóły w jaśniejszych obszarach zostają utracone.

Natomiast dla $T = 100$ (Rysunek 14) binaryzacja uzyskuje równowagę pomiędzy czernią a bielą i zachowuje szczegóły.

W aplikacji użytkownik ma możliwość dynamicznego ustawienia progu T za pomocą suwaka w interfejsie graficznym.

Wady i ograniczenia

Pomimo swojej prostoty i efektywności, binaryzacja ma pewne ograniczenia:

- Czulość na wartość progu: Wynik operacji binaryzacji zależy w dużym stopniu od doboru wartości T . Niewłaściwie dobrany próg może prowadzić do utraty istotnych szczegółów lub błędnej segmentacji. Można wykorzystać uautomatyzowaną selekcję progu: Algorytmy takie jak metoda Otsu mogą automatycznie wybrać optymalny próg T , minimalizując wpływ błędów użytkownika.
- Wpływ oświetlenia: Obrazy o nierównomiernym oświetleniu mogą dawać niejednoznaczne wyniki, ponieważ różnice w jasności mogą nie wynikać z cech obiektów, lecz z warunków rejestracji obrazu. Dla lepszych wyników można użyć binaryzacji adaptacyjnej: Polega na dynamicznym dostosowywaniu wartości progu T w zależności od lokalnych cech obrazu.

2.2 Filtry graficzne

Filtry graficzne są kluczowym elementem przetwarzania obrazów, umożliwiającym modyfikację, analizę oraz poprawę jakości obrazu poprzez uwzględnienie sąsiednich pikseli. W przeciwieństwie do operacji punktowych, filtry wykorzystują informacje o otoczeniu każdego piksela, co pozwala na osiągnięcie bardziej zaawansowanych efektów wizualnych i analitycznych.

Obraz cyfrowy, reprezentowany jako macierz pikseli, jest przekształcany za pomocą masek lub macierzy filtrów, które przetwarzają wartości w określonym obszarze wokół każdego piksela. Proces ten umożliwia realizację różnorodnych zadań, takich jak redukcja szumów, wygładzanie, wyostrażanie czy wykrywanie krawędzi.

2.2.1 Filtr uśredniający

Filtr uśredniający jest stosowany w celu redukcji szumów na obrazie i zmniejszenia różnic w jasności pomiędzy sąsiednimi pikselami, co skutkuje wygładzeniem obrazu.

Idea i metoda

Idea filtra uśredniającego opiera się na obliczeniu wartości każdego piksela jako średniej wartości pikseli znajdujących się w jego otoczeniu. Rozmiar otoczenia (tzw. maski) może wynosić np. 1×1 , 2×2 , 3×3 , 5×5 , 7×7 lub 9×9 . Dla maski 3×3 :

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix},$$

wartość nowego piksela $P(x, y)$ jest wyliczana według wzoru:

$$P(x, y) = \frac{1}{\sum_{i=-1}^1 \sum_{j=-1}^1 W(i, j)} \sum_{i=-1}^1 \sum_{j=-1}^1 W(i, j) \cdot I(x + i, y + j)$$

gdzie zakresy $i, j \in \{-1, 0, 1\}$ definiują maskę 3×3 , $I(x + i, y + j)$ to wartości pikseli w obrazie wejściowym, a $W(i, j)$ to wagi maski.

Dla maski 2×2 :

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix},$$

w której piksel $P(x, y)$ jest lewym górnym rogiem maski, wartość nowego piksela jest obliczana według wzoru:

$$P(x, y) = \frac{1}{\sum_{i=0}^1 \sum_{j=0}^1 W(i, j)} \sum_{i=0}^1 \sum_{j=0}^1 W(i, j) \cdot I(x + i, y + j)$$

gdzie zakresy $i, j \in \{0, 1\}$ definiują maskę 2×2 . Maski 2×2 są specyficzne, ponieważ przesuwane są w taki sposób, że piksel, dla którego liczona jest nowa wartość, jest zawsze lewym górnym rogiem.

Na Rysunek 16, Rysunek 17 i Rysunek 18 wyraźnie widać efekty działania filtra uśredniającego. Dla maski 3×3 (Rysunek 17) obraz jest bardziej wygładzony w porównaniu do oryginalnego obrazu (Rysunek 16). Natomiast dla maski 9×9 (Rysunek 18) wygładzenie jest znacznie bardziej widoczne, co jednak prowadzi do utraty części szczegółów obrazu.

Rozmiar maski można dynamicznie zmieniać za pomocą przycisków radiowych w interfejsie użytkownika.

Wady i ograniczenia

- Filtr uśredniający powoduje utratę detali obrazu, zwłaszcza na krawędziach lub w miejscach o dużych gradientach.



Rysunek 16: Oryginalny obraz



Rysunek 17: Obraz po użyciu filtra uśredniającego: maska 3×3



Rysunek 18: Obraz po użyciu filtra uśredniającego: maska 9×9

- Maski 2×2 mogą wprowadzać artefakty, ponieważ wartości są liczone tylko dla części otoczenia.
- Dla dużych rozmiarów maski proces może być obciążający obliczeniowo.

2.2.2 Filtr Gaussa

Filtr Gaussa ma zastosowanie podobne do filtra uśredniającego – jego głównym celem jest redukcja szumów i wygładzenie obrazu. Jest często używany w przetwarzaniu obrazów w celu eliminacji zakłóceń, zapewniając jednocześnie łagodne przejścia tonalne.

Idea i metoda

Filtr Gaussa opiera się na obliczeniu dla każdego piksela średniej ważonej wartości pikseli w jego otoczeniu. Dzięki temu piksele z większą odległością od centrum maski mają mniejszy wpływ na wynik, co prowadzi do bardziej naturalnego efektu wygładzenia. Maską wykorzystywaną w tym filtrze ma postać:

$$\begin{bmatrix} 1 & b & 1 \\ b & b^2 & b \\ 1 & b & 1 \end{bmatrix},$$

gdzie b to liczba naturalna definiująca intensywność wpływu sąsiednich pikseli. Wartość nowego piksela obliczana jest według wzoru:

$$P(x, y) = \frac{1}{\sum_{i=-1}^1 \sum_{j=-1}^1 W(i, j)} \sum_{i=-1}^1 \sum_{j=-1}^1 W(i, j) \cdot I(x + i, y + j),$$

gdzie zakresy $i, j \in \{-1, 0, 1\}$ definiują maskę 3×3 , $I(x + i, y + j)$ to wartości pikseli w obrazie wejściowym, a $W(i, j)$ to wagi maski.

Na rysunkach Rysunek 19, Rysunek 20 i Rysunek 21 przedstawiono efekty działania filtra Gaussa. Przy wartości $b = 2$ (Rysunek 20) obraz został wygładzony bardziej niż dla oryginalnego obrazu (Rysunek 19). Natomiast dla $b = 6$ (Rysunek 21) poziom wygładzenia jest mniejszy, co wynika z różnic w proporcjach wag maski.

Wartość parametru b można dynamicznie dostosowywać za pomocą przycisków radiowych w interfejsie użytkownika. Pozwala to na eksperymentowanie z różnymi ustawieniami filtra i dostosowanie go do konkretnych potrzeb użytkownika.



Rysunek 19: Oryginalny obraz



Rysunek 20: Obraz po użyciu filtra Gaussa: $b = 2$



Rysunek 21: Obraz po użyciu filtra Gaussa: $b = 6$

Wady i ograniczenia

- Filtr Gaussa może powodować utratę szczegółów w obrazie, zwłaszcza w przypadku małych obiektów i krawędzi, które mogą zostać rozmyte.
- W przypadku obrazów zawierających istotne wysokie częstotliwości (np. tekst lub cienkie linie) filtr może negatywnie wpłynąć na ich czytelność.
- Stosowanie filtra na dużych obrazach może być obciążające obliczeniowo, szczególnie przy dynamicznej zmianie parametrów i większych maskach.

2.2.3 Filtr wyostrzający

Filtr wyostrzający jest stosowany w celu podkreślenia szczegółów i krawędzi na obrazie poprzez wzmocnienie kontrastu między sąsiednimi pikselami.

Idea i metoda

W filtrze wyodrządzającym stosowana jest macierz wag (maska), która oblicza nową wartość piksela na podstawie wartości sąsiadujących pikseli.

Obliczanie nowego piksela wykorzystuje sumę ważoną wartości pikseli w oknie maski oraz ich wag. Dzięki temu krawędzie i obiekty o dużym kontraście stają się bardziej wyraźne, a płaskie obszary pozostają mniej zmienione. Wartość nowego piksela obliczana jest według wzoru:

$$P(x, y) = \frac{1}{\sum_{i=-1}^1 \sum_{j=-1}^1 W(i, j)} \sum_{i=-1}^1 \sum_{j=-1}^1 W(i, j) \cdot I(x + i, y + j),$$

gdzie zakresy $i, j \in \{-1, 0, 1\}$ definiują maskę 3×3 , $I(x + i, y + j)$ to wartości pikseli w obrazie wejściowym, a $W(i, j)$ to wagi maski.

Maski użyte w filtrze wyodrządzającym to:

$$\begin{aligned} \text{Maska 1: } & \begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}, & \text{Maska 2: } & \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}, \\ \text{Maska 3: } & \begin{bmatrix} -1 & -2 & -1 \\ -2 & 5 & -2 \\ -1 & -2 & -1 \end{bmatrix}, & \text{Maska 4: } & \begin{bmatrix} 0 & -1 & 0 \\ -1 & 9 & -1 \\ 0 & -1 & 0 \end{bmatrix}. \end{aligned}$$

Porównanie efektów masek:

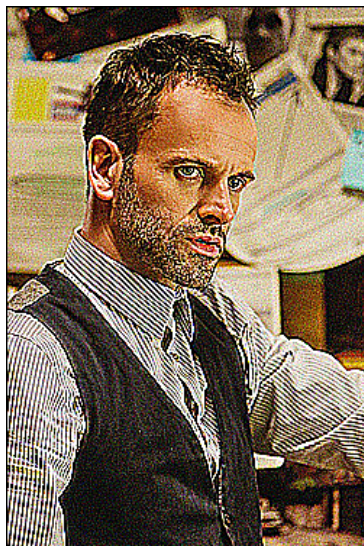
- **Maska 1:** Wysokie wzmocnienie centralnego piksela, silne wyodrządzanie obrazu, wyraźne detale i krawędzie.
- **Maska 2:** Mniej intensywne niż Maski 1, ale nadal wyraźne wyodrządzanie obrazu, łagodniejsze niż Maski 1.
- **Maska 3:** Wyodrzzanie z silniejszymi ujemnymi wartościami sąsiednich pikseli (-2), co prowadzi do bardziej wyrazistych krawędzi niż w Maski 2, ale z mniejszym kontrastem niż Maski 1.
- **Maska 4:** Bardzo podobna do Maski 1, ale z nieco mniejszym wpływem sąsiadujących pikseli, co daje bardziej subtelne wyodrzzanie.

Na rysunkach Rysunek 22, Rysunek 23 i Rysunek 24, Rysunek 25, Rysunek 26 przedstawiono efekty działania filtra wyodrzzającego. Obraz po użyciu Maski 1 Rysunek 23 — widzimy intensywne wyodrzzanie, z wyraźnymi, ostrymi krawędziami. Obraz po użyciu Maski 2 Rysunek 24

— efekt wyostrozania jest mniej intensywny, ale nadal wyraźny. Obraz po użyciu Maski 3 Rysunek 25 — mocniejsze wyostrozanie krawędzi i detali w porównaniu do Maski 2. Obraz po użyciu Maski 4 Rysunek 26 — intensywne wyostrozanie, ale mniej agresywne niż w Masce 1, z bardziej subtelnym efektem.



Rysunek 22: Oryginalny obraz



Rysunek 23: Obraz po użyciu filtra wyostrającego z maską 1



Rysunek 24: Obraz po użyciu filtra wyostrającego z maską 2



Rysunek 25: Obraz po użyciu filtra wyostrającego z maską 3



Rysunek 26: Obraz po użyciu filtra wyostrającego z maską 4

Wady i ograniczenia

- Zwiększone szумы – filtr wyostrający może wzmocnić nie tylko krawędzie, ale także niepożądane zakłócenia (szумы), szczególnie w obrazach o niskiej jakości.

- Ograniczona uniwersalność – filtr wyostrajający działa najlepiej w obrazach o wyraźnych krawędziach, ale może być mniej skuteczny w obrazach o niskim kontraście lub małych szczegółach.

2.2.4 Wykrywanie krawędzi - krzyż Robertsa

Krzyż Robertsa ma zastosowanie w analizie obrazów, szczególnie w zadaniach wykrywania krawędzi. Jego głównym celem jest identyfikacja zmian intensywności między pikselami, co umożliwia podkreślenie konturów obiektów na obrazie.

Idea i metoda

Krzyż Robertsa to prosty operator wykrywania krawędzi, który oblicza różnice w intensywności sąsiednich pikseli. Wykorzystuje dwie macierze wag, które operują na obrazach w skali szarości lub kolorowych. Podczas obliczeń stosowane są dwie macierze:

$$\text{Macierz wag 1: } \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad \text{Macierz wag 2: } \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}.$$

Działanie polega na konwolucji obu masek z obrazem, czyli obliczaniu iloczynów wartości pikseli z wagami w danym oknie. Wynik jest sumowany dla każdej maski, a następnie wartość krawędzi jest określana na podstawie ich połączenia. Metoda ta jest szczególnie skuteczna w przypadku krawędzi o dużym kontraście.

Na Rysunek 27, Rysunek 28 oraz Rysunek 29 przedstawiono wyniki działania operatora krzyża Robertsa. Na Rysunek 28 widać podkreślenie krawędzi, jednak pojawiają się także kolorowe artefakty wynikające z niezależnego przetwarzania kanałów RGB. Natomiast na Rysunek 29, dzięki wcześniejszemu przekształceniu obrazu do odcieni szarości, krzyż Robertsa skutecznie podkreślił krawędzie, zachowując naturalny wygląd bez dodatkowych efektów.

Wady i ograniczenia

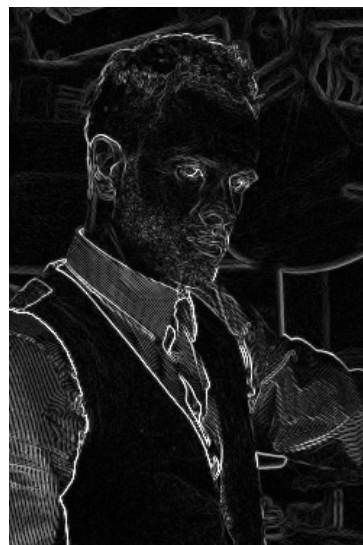
- Niska odporność na szumy — Krzyż Robertsa może wzmacniać szumy w obrazach, co prowadzi do wykrycia fałszywych krawędzi. Aby zredukować wpływ szumów na wynik można wcześniej zastosować filtr odszumiający.
- Dziwne wyniki dla obrazów kolorowych — dla obrazów RGB kanały są przetwarzane osobno, co może powodować niespójności i nienaturalne rezultaty. Aby temu zapobiec można najpierw zrobić konwersję do odcieni szarości.



Rysunek 27: Oryginalny obraz



Rysunek 28: Obraz po użyciu krzyża Roberta



Rysunek 29: Obraz został najpierw przekształcony do odcieni szarości, a następnie użyto krzyża Roberta

2.2.5 Wykrywanie krawędzi - operator Sobela

Operator Sobela ma zastosowanie w analizie obrazów, szczególnie w zadaniach wykrywania krawędzi. Jego głównym celem jest identyfikacja zmian intensywności między pikselami w kierunkach poziomym i pionowym, co umożliwia podkreślenie konturów obiektów na obrazie.

Idea i metoda

Operator Sobela wykorzystuje dwie macierze wag, które służą do obliczania gradientów w obu kierunkach:

$$\text{Macierz dla kierunku X: } \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad \text{Macierz dla kierunku Y: } \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

Metoda polega na konwolucji tych masek z obrazem wejściowym w celu obliczenia gradientów intensywności pikseli w kierunkach X i Y. Wynik gradientu dla każdego piksela obliczany jest jako:

$$G = \sqrt{G_x^2 + G_y^2},$$

gdzie G_x i G_y są gradientami w kierunkach X i Y. Operator Sobela podkreśla krawędzie, które mają największe zmiany intensywności, dzięki czemu można skutecznie wyodrębnić kontury obiektów.

Na Rysunek 30, Rysunek 31 oraz Rysunek 32 przedstawiono wyniki działania operatora Sobela. Na Rysunek 31 widać podkreślenie krawędzi, jednak pojawiają się także kolorowe artefakty wynikające z niezależnego przetwarzania kanałów RGB. Natomiast na Rysunek 32, dzięki wcześniejszemu przekształceniu obrazu do odcieni szarości, operator Sobela skutecznie podkreślił krawędzie, zachowując naturalny wygląd bez dodatkowych efektów.



Rysunek 30: Oryginalny obraz



Rysunek 31: Obraz po użyciu operatora Sobela



Rysunek 32: Obraz został najpierw przekształcony do odcieni szarości, a następnie użyto operatora Sobela

Wady i ograniczenia

- Operator Sobela wykazuje niską odporność na szumy, które mogą prowadzić do fałszywego wykrycia krawędzi. Aby redukować wpływ szumów na wynik można wcześniej zastosować filtr odszumiający.
- Wyniki dla obrazów kolorowych (np. Rysunek 31) mogą być niespójne, ponieważ każdy kanał (R, G, B) jest przetwarzany osobno, co prowadzi do kolorowych artefaktów. Aby temu zapobiec można najpierw zrobić konwersję do odcieni szarości

2.2.6 Możliwość tworzenia dowolnych wag filtrów

Aplikacja została wzbogacona o możliwość ręcznego wprowadzania dowolnych wag filtrów przez użytkownika. Funkcja ta umożliwia definiowanie własnych macierzy wag, które są następnie wykorzystywane do przetwarzania obrazu.

Idea i metoda

Użytkownik wprowadza macierz wag w polu tekstowym w odpowiednim formacie (np. `[[ -1,`

$0, 1], [0, 1, 0], [-1, 0, 1]$). Po zatwierdzeniu aplikacja przekształca tekst na macierz, a następnie oblicza wartość piksela według wzoru:

$$P(x, y) = \frac{1}{\sum_{i=-1}^1 \sum_{j=-1}^1 W(i, j)} \sum_{i=-1}^1 \sum_{j=-1}^1 W(i, j) \cdot I(x + i, y + j),$$

gdzie zakresy $i, j \in \{-1, 0, 1\}$ definiują maskę 3×3 , $I(x + i, y + j)$ to wartości pikseli w obrazie wejściowym, a $W(i, j)$ to wagi maski.

Dzięki tej funkcji użytkownicy mają pełną kontrolę nad definiowaniem operacji przetwarzania obrazu, co pozwala na eksperymentowanie z różnymi efektami filtrów.

2.3 Wykresy

Wykresy są istotnym narzędziem w analizie obrazów, umożliwiającym wizualizację różnorodnych właściwości obrazu i jego charakterystyki. Dzięki nim można w prosty sposób zidentyfikować rozkład jasności, barw czy inne specyficzne cechy, które mogą być trudne do uchwycenia za pomocą samych danych liczbowych.

Obraz cyfrowy, reprezentowany jako macierz pikseli, może być analizowany poprzez różne techniki graficzne, takie jak histogramy jasności i kolorów, czy projekcje obrazu w kierunkach poziomym i pionowym. Te metody pozwalają na intuicyjne przedstawienie struktury i zawartości obrazu, wspomagając dalsze procesy przetwarzania lub analizy.

2.3.1 Histogram jasności

Histogram jasności obrazu to narzędzie służące do wizualizacji rozkładu jasności w obrazie.

Idea i metoda

Każdy piksel obrazu jest przypisywany do odpowiedniego przedziału jasności, a histogram przedstawia liczbę pikseli w każdym z tych przedziałów.

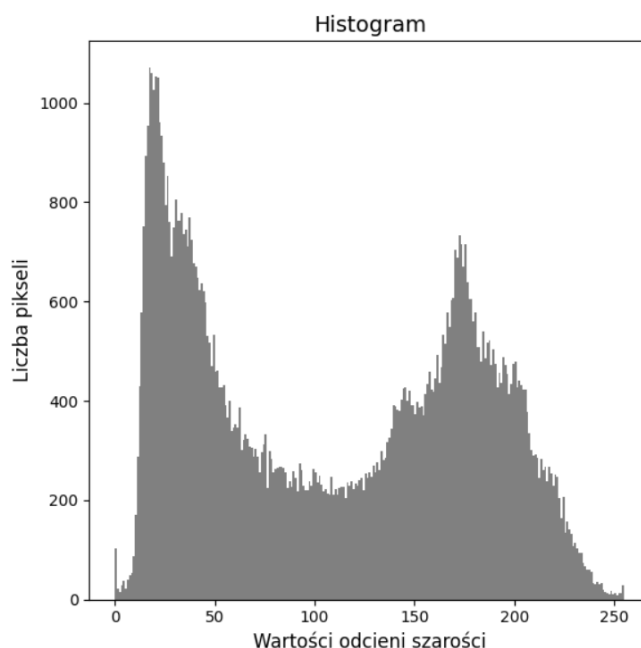
Metoda polega na:

1. Konwersji obrazu na skalę szarości, co upraszcza analizę poprzez redukcję kanałów barwnych.
2. Wyodrębnieniu wartości odcieni szarości z macierzy obrazu.
3. Grupowaniu pikseli w przedziały jasności (tzw. bins).
4. Wizualizacji liczby pikseli w każdym przedziale za pomocą słupków histogramu.

Na Rysunek 33 i Rysunek 34 przedstawiono oryginalny obraz oraz jego histogram jasności. Histogram w Rysunek 34 wskazuje na dużą liczbę ciemnych pikseli o wartościach jasności w przedziale $[0, 50]$, a także znaczną ilość jaśniejszych pikseli w zakresie $[150, 200]$.



Rysunek 33: Oryginalny obraz



Rysunek 34: Histogram jasności oryginalnego obrazu

2.3.2 Histogram kolorów

Histogram kolorów pozwala na analizę rozkładu intensywności pikseli w każdym z trzech kanałów barwnych obrazu RGB: czerwonym, zielonym i niebieskim. Dodatkowo, może być uzupełniony o histogram jasności, który reprezentuje skalę szarości obrazu.

Idea i metoda

Metoda polega na wyodrębnieniu wartości każdego kanału barwnego (R, G, B) z macierzy obrazu oraz grupowaniu pikseli w przedziały intensywności. Następnie dla każdego kanału generowany jest oddzielny histogram, który wizualizuje liczebność pikseli w różnych zakresach intensywności.

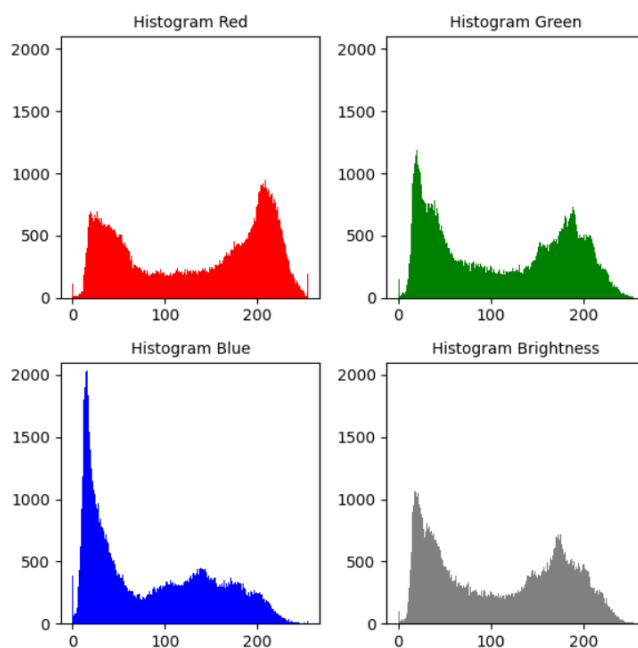
Na Rysunek 35 i Rysunek 36 przedstawiono oryginalny obraz oraz jego histogram jasności. Na Rysunek 36 widać, że obraz zawiera bardzo mało koloru niebieskiego, a dominują kolory zielony i czerwony.

2.3.3 Projekcja pozioma

Projekcja pozioma obrazu to technika analizy obrazu polegająca na sumowaniu wartości jasności pikseli w poszczególnych wierszach macierzy reprezentującej obraz. Dzięki temu możliwe jest



Rysunek 35: Oryginalny obraz



Rysunek 36: Histogramy kolorów i jasności oryginalnego obrazu

zidentyfikowanie poziomych struktur w obrazie, takich jak linie, tekst czy obszary o wyraźnych zmianach jasności.

Idea i metoda

Projekcja pozioma działa w następujący sposób:

1. Konwertuje obraz do skali szarości, aby uprościć analizę wartości jasności.
2. Dla każdego wiersza obrazu oblicza sumę jasności wszystkich pikseli w tym wierszu.
3. Tworzy wykres liniowy, gdzie na osi X przedstawione są sumy jasności w poszczególnych wierszach, a na osi Y numer wiersza.
4. Obszar pod wykresem jest wypełniony kolorem, aby uwidocznić rozkład jasności w poziomie.

2.3.4 Projekcja pionowa

Projekcja pionowa obrazu to technika analizy obrazu polegająca na sumowaniu wartości jasności pikseli w poszczególnych kolumnach macierzy reprezentującej obraz. Dzięki temu możliwe jest zidentyfikowanie pionowych struktur w obrazie, takich jak linie, tekst czy obszary o wyraźnych zmianach jasności.

Idea i metoda

Projekcja pionowa działa w następujący sposób:

1. Konwertuje obraz do skali szarości, aby uprościć analizę wartości jasności.
2. Dla każdej kolumny obrazu oblicza sumę jasności wszystkich pikseli w tej kolumnie.
3. Tworzy wykres liniowy, gdzie na osi X przedstawione są sumy jasności w poszczególnych kolumnach, a na osi Y numer kolumny.
4. Obszar pod wykresem jest wypełniony kolorem, aby uwidocznić rozkład jasności w pionie.

2.3.5 Projekcja pozioma i pionowa

Opcja **Projekcja pozioma i pionowa** służy do wyświetlania zarówno projekcji poziomej, jak i pionowej obrazu. Umożliwia ona analizę rozkładu jasności w obrazie zarówno w kierunku poziomym, jak i pionowym, co pozwala na łatwiejsze rozpoznawanie struktur w obrazie, takich jak linie, teksty czy obszary o wyraźnych zmianach jasności.

Funkcja ta generuje układ, w którym:

1. W pierwszym oknie (po lewej stronie) wyświetlany jest oryginalny obraz.
2. W drugim oknie (po prawej stronie, u góry) wyświetlana jest projekcja pozioma obrazu. Jest to wykres liniowy, na którym oś X przedstawia sumy jasności dla poszczególnych wierszy obrazu, a oś Y reprezentuje numery wierszy.
3. W trzecim oknie (po lewej stronie, na dole) wyświetlana jest projekcja pionowa. To wykres liniowy, na którym oś X przedstawia sumy jasności dla poszczególnych kolumn obrazu, a oś Y reprezentuje numery kolumn.

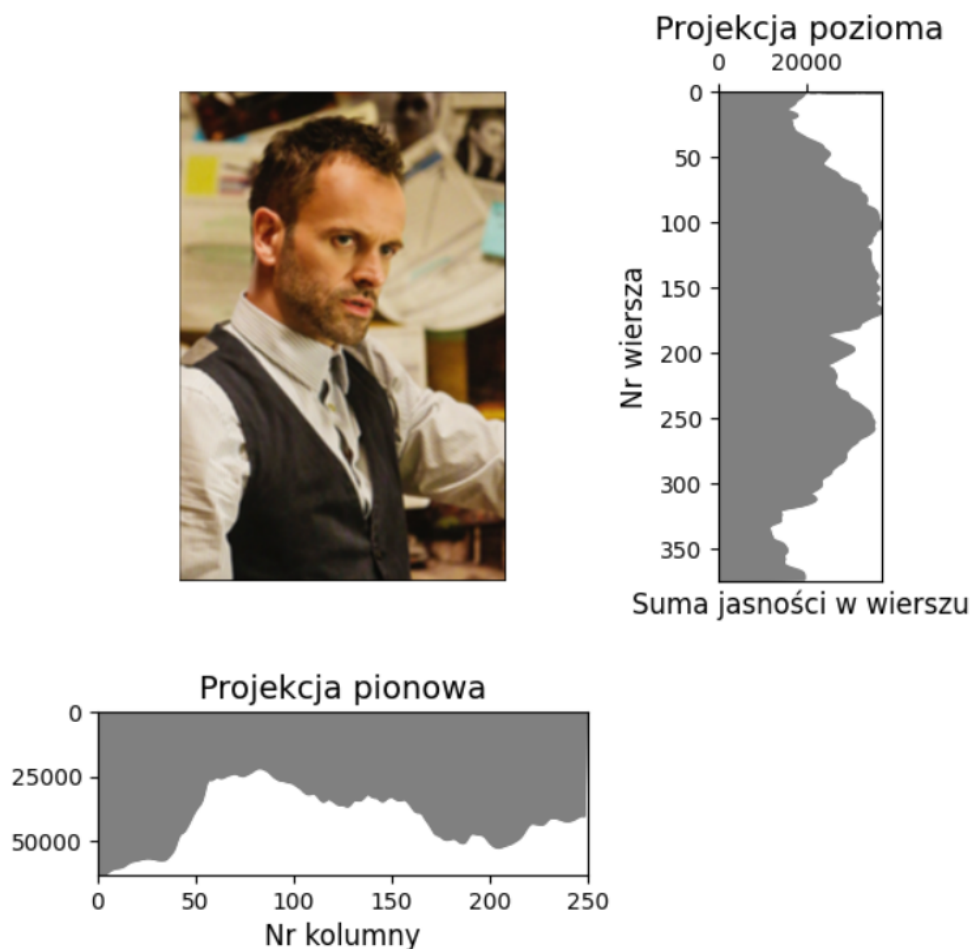
Rysunek 37 przedstawia projekcje poziomą i pionową obrazu.

2.4 Inne

2.4.1 Wczytanie obrazu z pliku

Wczytanie obrazu z pliku to podstawowa operacja w przetwarzaniu obrazów, umożliwiającą użytkownikowi załadowanie obrazu z dysku do dalszej analizy.

W zaimplementowanej metodzie proces ten odbywa się za pomocą interfejsu graficznego, który umożliwia wybór pliku z systemu plików, używając narzędzia `filedialog`. Użytkownik wybiera obraz w jednym z obsługiwanych formatów (np. `.png`, `.jpg`, `.jpeg`, `.bmp`), a następnie obraz jest wczytywany do pamięci przy użyciu biblioteki `PIL` (Python Imaging Library).



Rysunek 37: Oryginalny obraz i jego projekcja pozioma oraz pionowa

Po załadowaniu obrazu, jest on konwertowany na macierz, co pozwala na łatwą manipulację jego danymi, takimi jak modyfikacja pikseli czy obliczenia statystyczne. Następnie jest on wyświetlany w lewym panelu w aplikacji.

2.4.2 Zapisu pliku przetworzonego

Zapisanie przetworzonego obrazu do pliku jest istotnym krokiem w przetwarzaniu obrazów, umożliwiającym użytkownikowi zapisanie wyników jego pracy w formie pliku na dysku.

W zaimplementowanej metodzie użytkownik może wybrać lokalizację i nazwę pliku, w którym zapisany zostanie obraz. Do tego celu wykorzystano narzędzie `filedialog`, które pozwala na wybranie miejsca docelowego oraz format pliku (np. `.png`, `.jpg`, `.bmp`).

Po wybraniu lokalizacji pliku, metoda konwertuje macierz pikseli, reprezentującą przetworzony obraz, z powrotem na obraz przy pomocy biblioteki PIL (Python Imaging Library). Następnie obraz jest zapisywany na dysku.

Aby dostarczyć użytkownikowi informację o zakończeniu procesu zapisu, wyświetlany jest

komunikat tymczasowy, który po kilku sekundach znika.

2.5 Podsumowanie

Implementacja funkcji operacji na pikselach oraz filtrów graficznych w ramach tego projektu okazała się skuteczna, ale wymaga odpowiedniego dopasowania parametrów do specyfiki przetwarzanego obrazu. W trakcie testów zauważono pewne trudności związane z różnymi formatami plików obrazów oraz ich rozdzielczościami, zwłaszcza w przypadku obrazów o dużych rozdzielczościach, które wymagały większych zasobów pamięci. Z tego powodu niektóre operacje, jak filtracja, mogły mieć wolniejszy czas wykonania.

Każda z metod działa najlepiej w określonych warunkach, a ich efektywność może być ograniczona przez czynniki takie jak jakość obrazu, jego rozdzielczość czy obecność szumów. Mimo tych wyzwań, aplikacja stanowi użyteczne narzędzie do przetwarzania obrazów, oferując intuicyjny interfejs oraz umożliwiając użytkownikowi łatwe wykonywanie podstawowych operacji graficznych w sposób przejrzysty i interaktywny.